

**NATURAL LANGUAGE PROCESSING FOR ROBOTIC NAVIGATION
THROUGH UNKNOWN ENVIRONMENTS**

by

Olivia Piazza

A thesis submitted to

Sonoma State University

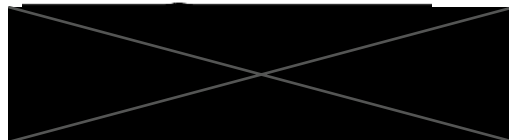
in partial fulfillment of the requirements

for the degree of

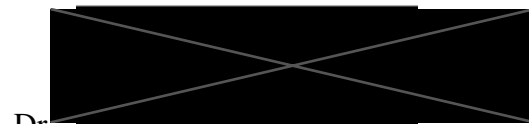
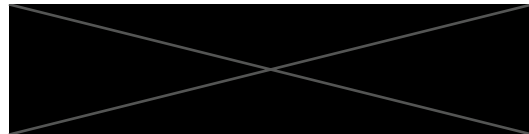
MASTER OF SCIENCE

in

Computer and Engineering Science



r. u r res a, a r



Dr.

12/11/2020

Date

Copyright 2020

By Olivia Piazza

AUTHORIZATION FOR REPRODUCTION OF MASTER'S THESIS

I grant permission for the print or digital reproduction of parts of this thesis. without further authorization from me, on the condition that the person or agency requesting reproduction absorb the cost and provide proper acknowledgment of authorship.

Date: December 11th, 2020

Signature:



Street Address



City, State, Zip

I would like to dedicate this thesis to all the students and researchers who have strove for excellence during these pandemic of times. And also to my husband, Tristan, and my son John, without whom I might not have had the strength to strive myself. Thank you.

ABSTRACT

Purpose of the Study: The purpose of this study is to explore the efficacy of natural language processing in generalizing the navigation abilities of robotic agents from known environments to unknown environments.

Procedure: To determine the effect of natural language processing on generalizing robotic navigation to unknown environments, baseline decision-making methods of student-forcing and teacher-forcing were used in a language interpretation and navigation context. Next, the language interpretation network was modified to include a human-to-machine translated network based on an instruction generating network. The efficacy of the machine-translated-network was scored using a Bilingual Evaluation Understudy (BLEU) metric. Navigation agents then repeated navigation tasks using the modified language interpretation network and navigation metrics were collected including navigation error, rate of success of the navigation agent stopping to within 3 m of the goal, and rate of success of the navigation agent's trajectory coming within 3 m of the goal at any point in the trajectory.

Findings: The use of natural language in the form of instructional augmentation improved each baseline method used by a margin of 12% or greater in all navigation metrics. In methods which only use modified instructions during decision making processes, such as student-forcing navigation agents, instructional augmentation improved the baseline method by a margin of 27% or greater in all navigation metrics. This results demonstrates that the use of language augmentation for semi-supervised navigation agents is an effective method to help generalize navigation from known to unknown environments.

Conclusions: Using natural language augmentation for instruction interpretation consistently shows an improvement of navigation success rates and reduction of navigation errors when generalizing robotic navigation from seen to unseen environments, especially when using semi-supervised navigation methods.

Chair: _____

Signature

Date: _____

MSCES Program: Engineering Science
Sonoma State University

ACKNOWLEDGMENTS

I would like to acknowledge first and foremost my advisor Dr. Sudhir Shrestha for his patience, flexibility, and stride in the face of challenge through these unprecedented times. His vision has been invaluable and his guidance critical to the success of this project. I would also like to acknowledge my committee members, Dr. Mohamed Salem and Dr. Nansong Wu for their expertise and contributions to the development of this thesis.

I would like to acknowledge Dr. Shahradsad Jamshidi and my colleague Kirk Duran for guidance and inspiration when studying and developing the machine learning portions of the project, especially to Kirk for our discussions around the intricacies and implementation of the instruction augmentation network. Thank you friend. Your intellect and resourcefulness will serve you well.

Finally, I would like to acknowledge Sonoma State University and the Koret Foundation for funding this thesis through the Sonoma State Research Award and the Sonoma State Mini-Grant program.

TABLE OF CONTENTS

Acknowledgments	vi
List of Tables	x
List of Figures	xi
List of Acronyms	xiii
Chapter 1: Introduction and Background	1
1.1 Motivations	1
1.2 Project Overview	2
1.3 Prior Work	2
1.3.1 Navigation Without Learning Algorithms	3
1.3.2 Navigation Using Learning Algorithms	3
1.3.3 Navigation Through Unknown Environments	4
1.3.4 Language and Vision-Based Navigation	4
1.3.5 Active Learning Navigation	5
Chapter 2: The Navigation Agent	7
2.1 Theory	7
2.1.1 Neural Networks	7

2.1.2	Convolutional Neural Networks	11
2.1.3	Recurrent Neural Networks	13
2.1.4	Teacher Forcing vs Student Forcing	17
2.2	Language Augmentation and Instruction	18
2.3	Action-In-Image Space	18
2.4	Navigation Prediction	19
Chapter 3: Methodology		20
3.1	The Data Set	20
3.2	Experimental Setup	21
3.2.1	Training	21
3.2.2	Implementation	22
3.3	Evaluation Metric	23
3.3.1	Bleu: Bilingual Evaluation Understudy	23
3.3.2	Navigation Metrics	24
Chapter 4: Results and Analysis		25
4.1	Language Training Results	25
4.2	Navigation Results	25
Chapter 5: Comparison to Prior Work		28
5.1	Navigation Models	28
5.1.1	Human Baseline	28
5.1.2	Bring Me A Spoon	29

5.1.3	Reinforced-Learning for Planned-Ahead Navigation	30
5.2	Augmentation Models	30
5.2.1	Speaker-Follower Model	30
Chapter 6:	Conclusion	31
6.1	Future Work	31
Appendices		33
	Appendix A: Navigation Example	34
References		35

LIST OF TABLES

4.1	Summary of Results of Methods. As baselines, both random and greedy searches were used on identical validation/test sets as the teach-forced/student-forced models as well as the augmented instruction teacher-forced/student-forced models. Metrics collected included navigation error (Navigation Error (NE)), in meters, success rate (Success Rate (SR)), (with a cutoff of 3 m from goal), and oracle success rate (Oracle Success Rate (OSR)), (cutoff of 3 m from goal at any point in the trajectory). A low navigation error is more desirable, whereas high success rates and oracle success rates are more desirable.	27
5.1	Comparison to Other Work. The human baseline is a series of human-generated path plans, whereas the remaining methods are all machine learning based path planning models. Reinforced Plan-Ahead (RPA) and Bring Me A Spoon (BMAS) are both models which emphasize navigation model development whereas the speaker-follower model utilized data input augmentation as well as navigation model development.	28

LIST OF FIGURES

2.1	Structure of a fully connected feed forward neural network. The input layer is comprised of neurons that take in external data then feed that data to every neuron in the next, hidden layer. Each neuron from the hidden layer then processes the input neuron information then feed the data to the output layer. The output layer can be of a similar form as the input layer, such as style transfer networks, or can be of discrete outputs, such is the case with classification.	8
2.2	Computational structure of a single neuron. The neuron takes in one to several inputs, x_i , then calculates an associated weight for each input node. The weights are then summed together and a bias parameter, b , is added. The result is fed through an activation function and a single output is produced. This output could feed into another neuron, or the output layer of the network.	9
2.3	Back propagation and convergence. A loss function dependent upon the values of the weights calculated by the neural net, $J(w)$ is optimized using gradient descent until ideally a global cost minimum is found.	11
2.4	Architecture of a convolutional neural network. The network begins with an input layer, usually image data, then extracts feature vectors via convolution. The convolutional layer feeds into a pooling layer to limit the sensitivity to distortions. Finally, the output feature vectors are fed to a fully connected layer where the mapping of convolutional data to desired outputs occur.	12
2.5	Comparison of various Residual Network (ResNET) architectures. ResNET-152 architecture was used for the image processing in this thesis.	13
2.6	Unfolded Recurrent Neural Network (RNN) architecture. Cells, often referred to as “memory cells”, are a function of an input parameter x_t and some previous, hidden parameter, s_t . Cells input parameters are then fed through a $\tanh$ activation function to output some value at the current step, o_t . Image source: [34]	14

2.7	Structure of a Long Short-Term Memory, Long-Short Term Memory (LSTM), block. LSTMs are a function of the previous, hidden input parameter, s_{t-1} , some input parameter x_t , and the previous candidate cell, c_{t-1} . The hidden and input parameters must go through a forget, update, and output gate before producing an output for the current time step, o_t , as well as the next candidate cell, c_t and hidden parameter, s_t	15
2.8	Example of corrective action in teacher forcing. (a) Without teacher forcing, network predictions can deviate greatly from what is expected. (b) With teacher forcing, network predictions can be corrected at every step. Image source: [35]	17
3.1	Example of a Matterport 3D scene both with and without panoramic view. In the original data set for the room-to-room navigation task, all images are stitched together to create a panoramic image. The panoramic image is then embedded with navigational path data as well as camera position data.	21
3.2	A dollhouse model example of a navigable scenery created by Matterport 3D. In these models, meshes are rendered as well as room segmentation to allow for easier object and room identification. [40].	22
4.1	Loss function and BLEU metric score, measured in percent precision, for the instruction augmentation network.	25
A.1	Example path of the following instruction: “ <i>Go up the stairs and turn right. Enter the first door on the left. Stop at the bathtub.</i> ”	34

LIST OF ACRONYMS

BLEU Bilingual Evaluation Understudy

BMAS Bring Me A Spoon

CNN Convolutional Neural Network

LSTM Long-Short Term Memory

NE Navigation Error

OSR Oracle Success Rate

R2R Room-to-Room

ReLU Rectified Linear Unit Function

ResNET Residual Network

RGB-D RGB-Depth Image

RNN Recurrent Neural Network

RPA Reinforced Plan-Ahead

S2S Sequence-to-Sequence

SR Success Rate

CHAPTER 1

INTRODUCTION AND BACKGROUND

1.1 Motivations

Over the past decade, deep learning networks have been trained to recognize speech, translate text between languages at high level of performance, and even create art; with applications of deep learning networks to real-world problems becoming increasingly ubiquitous. For the field of robotics, however, deep reinforcement learning algorithms often require millions of attempts before learning to solve a task and are thus too inefficient to be readily deployable –especially in the area of navigation [1]. With the development of deep exploration algorithms and semi-supervised methods, however, machine learning is quickly approaching efficient solutions for optimized path navigation in unknown environments. Furthermore, being able to interface with a navigation agent on a human-like level seems like a logical advancement in the cross-section of machine learning and robotics and is achievable given the current development of the field of machine learning.

Combining the two by developing a navigation agent that can interpret human-generated instructions to reach some goal location in an unknown environment is not a trivial task. Consider the following set of instructions: *“Go up the stairs and turn right. Enter the first door on the left. Stop at the bathtub.”* To follow these instructions, the agent must learn what each object is, their relation to the environment, then determine its relative relation to the object, before finally determining the next appropriate, navigable step. Identifying things like “stairs” and “bathtubs” can be done using already developed object identification networks [2]. However, when instructions become too lengthy, common language interpretation algorithms become less efficient, leading wrong conclusions to be drawn or the network not converging altogether. Thus, it is beneficial for the navigation agent to con-

dense the instruction input and in the process, determine which information is most relevant and which can safely be discarded. Ground natural language as well as an auto-encoded machine translation network can provide the mechanisms through which information's relevancy can be determined and form a basis of machine-understanding built independently from the navigation agent's environment.

1.2 Project Overview

This thesis aims to explore the effect of natural language processing on language-and-vision robotic navigation through unknown environments. More specifically, this thesis aims to characterize the efficacy of natural language on language-and-vision style navigation when generalizing from previously seen to previously unseen data sets through auto-encoded instructional augmentation.

The remainder of this thesis is divided into five sections, delineated as follows: Section 2 describes the navigation agent in detail, including the theory underlying the agent and the structure of the agent; Section 3 provides an explanation of the data set used to train the agent as well as the experimental setup and the metrics used to evaluate the agent's performance; Section 4 contains the results and discussion; Section 5 will compare previous results with the results obtained using the methods described in this paper, highlighting the differences in outcomes as well as any limitations of the current method; finally, Section 6 will briefly conclude the outcomes of our experiments and outline any potential future work.

1.3 Prior Work

Contemporary approaches to autonomous navigation are dictated by the scope of the environmental data available to the navigation agent, namely whether the data is global or local. Navigation based on global environmental data conducts path searches in an algorithmic manner with precise environmental states known at all times. Such algorithms are

generally sampling-based path planning algorithms, e.g., random tree searches, or graph searches [3]. Both approaches are well optimized, efficient, and fully solved. Both approaches, however, require full state information of the environment to be known and are thus not valid approaches to the unknown environment navigation problem.

1.3.1 Navigation Without Learning Algorithms

Navigation agents without learning capabilities usually lack the flexibility necessary to adapt to unknown environments, especially if the environment is non-static. These algorithms include Artificial Potential Fields [4] or Velocity Obstacle (VO) [5] and calculate motion in real-time based on local environment data detected through the use of sensors. These algorithms are generally reactionary algorithms and often lead to poor performance in tasks that contain any ambiguity, such as interpreting human-generated instructions.

1.3.2 Navigation Using Learning Algorithms

Deep learning methods [6] often deploy extensive neural networks to interpolate patterns from previously explored environments. This requires a large data set, hand-labelled with optimized path choices. This method is time-consuming and does not generalize well to unknown, unlabelled data sets. Thus deep learning is often not applied to solve the type of problem navigation through unknown environments presents. Deep reinforced learning methods provide one alternative since the algorithm's success is not dependent upon label path data. Deep reinforced learning is predicated on some value-based or policy-based reward system and learns from the interactions the agent with the environment [1] and is highly efficient in solving simple navigation tasks such as navigation through mazes.

When presented with an incomplete and noisy environments to traverse, deep reinforced learning algorithms' success often do not generalize well, being easily trapped in challenging terrain such as long corridors [3]. Furthermore, the emphasis on rewards, which are naturally sparsely distributed in navigation environments, leads to slow learning rates and

possibly non-convergence. Thus deep reinforced learning algorithms do not generalize well to the problem of navigating unknown environments.

1.3.3 Navigation Through Unknown Environments

In order to create a successful navigation agent that generalizes well from known to unknown environments, it is necessary to create a robust algorithm which is fairly independent of supervision. Better exploration networks have generalized well to this regime, often assuming previous states when confusion arises and choosing an alternate path [7, 8]. Such a method requires many iterations of partial path exploration, and the length of traversed trajectories tend to be much longer than necessary, without much hope for improvement upon this metric when generalized to unknown environments. An alternative approach is to rely on semi-supervised navigation algorithms with data augmentation [9, 10, 11]. Though this approach suffers from human-bias due to the nature of the generation of augmented data, it does not suffer from the same inefficiency present in previously mentioned techniques and generalizes well to unknown environments.

1.3.4 Language and Vision-Based Navigation

Instruction-based navigation tasks that use vision and language systems to perform robotic navigation have been investigated extensively, especially in simulated environments [12, 13, 14], and more recently, photo-realistic environments [12, 15]. The vision-and-language navigation task has received significant attention, with several research groups utilizing image augmentation [10], better exploration and localization strategies [16, 17, 8], or generating more dynamic responses to diverse training environments [9], to create efficient navigators. Though these techniques have achieved more powerful and autonomous agents, a lack of interaction between human and robot would reduce these techniques to guess work if the robot were to be provided quasi-ambiguous instructions or become stuck in its navigation decision process.

Dialogue as a medium for human robot interfacing has been long sought after in the field of artificial intelligence [18], especially the use of dialogue to complete physical tasks [19, 20, 21]. Presented May of 2019, in the IEEE International Conference on Robotics and Automation, Thomason et al. demonstrated an end-to-end pipeline for mapping natural language commands to discrete robot actions [22], allowing the robot to adapt continually to new language constructs and perceptions. Furthermore, clarification dialogues were used to improve language parsing and concept grounding, but only before navigation tasks and therefore do not allow for human-robot interaction should confusion arise during the task. Later work by the same group [23] proposed to integrate the human-robot interaction by introducing a human-to-human dialogue data set for the navigation task into their vision-and-language framework.

1.3.5 Active Learning Navigation

Active learning is a special type of machine learning algorithm that can interactively query an information source to label new data points with the desired outputs [24]. For robotics, the first work that utilized human feedback for task learning was reported by Christiano et al [25], in which the feedback was implemented in an off-policy manner. Because this method is implemented as an offline method, it is not obvious if the human interaction was beneficial to the agent. Online methods that have been implemented only encourages the agent to ask for help when the expected state differs from the actual state, that is, when the robot cannot complete the task due to some obstacle in the actual state [26, 27, 28]. In this situation, the human is expected to repair the condition which caused the environment to fail, a method which does not generalize well for most learning situations.

For the vision-and-language navigation task, many groups have implemented active learning models in the navigation decision network with success [15, 10]. These models, known as teacher-forcing and student-forcing, rely on true state or previous state information to make predictions on the optimized trajectories and, specifically with student-forcing

algorithms, generalize well to unknown environments due to their semi-supervised nature. It is thus expected that a combination of language grounded instruction augmentation, as well as the aforementioned active learning models will perform well, independent of the known or unknown state of the environment.

CHAPTER 2

THE NAVIGATION AGENT

2.1 Theory

For a successful implementation of a language-and-vision based autonomous navigation agent, several advanced computational components are needed. For this particular implementation, the navigation agent consisted of three major modules: (1) Language interpretation model, to include natural language instruction generation; (2) Image analysis in the form of a convolutional neural network for object identification and room segmentation; (3) Navigation predication which intakes information from both the language interpretation model and the image analysis model to decide which navigable next step is best, including nuances about what information to supplant especially when stuck in the form of teacher forcing or student forcing.

The basic structural element of all these models is a machine learning model known as a neural network. A powerful class of non-linear function, neural networks can be implemented in different forms as well as attached to each other to create deep learning models that solve complex problems with minimal prior knowledge, supervision and implementation detail.

2.1.1 Neural Networks

A neural network is a dynamic computational network designed to transform an input into a desired output, generally of a different form. Inspired by the biological structures of the brain, neural networks consist of a neuron, or node, connected to other neurons with each neurological link having an associated weight that determines the strength of influence from one neuron to the other [29].

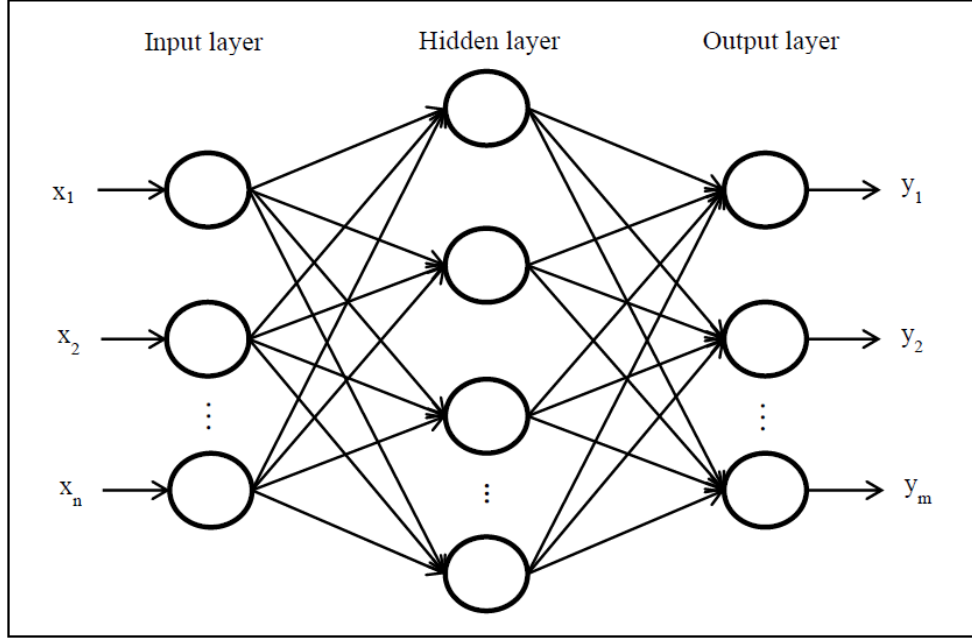


Figure 2.1: Structure of a fully connected feed forward neural network. The input layer is comprised of neurons that take in external data then feed that data to every neuron in the next, hidden layer. Each neuron from the hidden layer then processes the input neuron information then feed the data to the output layer. The output layer can be of a similar form as the input layer, such as style transfer networks, or can be of discrete outputs, such is the case with classification.

Neural networks are generally organized into multiple layers with neurons of one layer connected to neurons of the previous and next layers, but typically not the same layer (Figure 2.1). The input of each neuron can either be feature data extracted from some sample source, such as an image, or can be the outputs of other neurons. The layer that takes in data from the sample source is known as the input layer, the layers in the middle are called hidden layers, and the final layer is known as the output layer.

To find the outputs of a neuron, the weights of all inputs to the neuron is summed, then a bias term is added to this sum, more succinctly written as:

$$f(x) = \sum w_i x + b \quad (2.1)$$

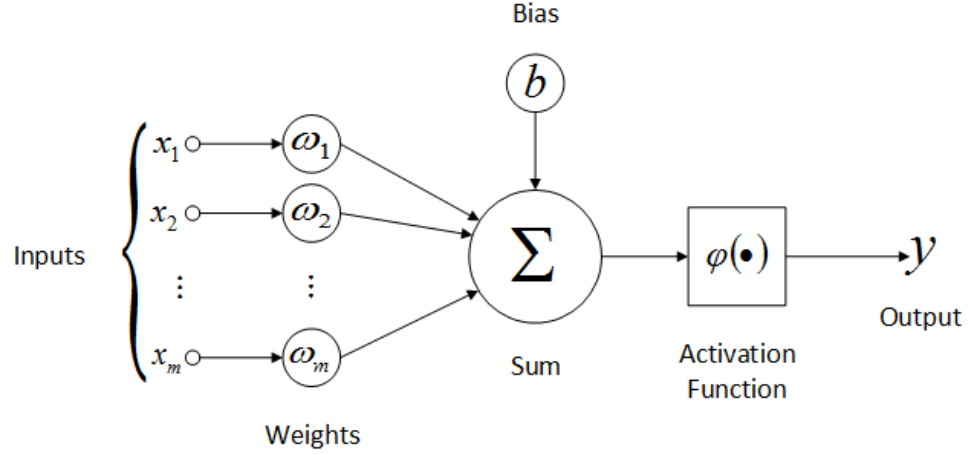


Figure 2.2: Computational structure of a single neuron. The neuron takes in one to several inputs, x_i , then calculates an associated weight for each input node. The weights are then summed together and a bias parameter, b , is added. The result is fed through an activation function and a single output is produced. This output could feed into another neuron, or the output layer of the network.

Where the function is required to output 0 or 1 according to the following:

$$f(x) = \begin{cases} 0 & \text{if } wx + b \leq 0 \\ 1 & \text{if } wx + b > 0 \end{cases} \quad (2.2)$$

Each neuron has a function like this associated with it known as the propagation function; this particular function is known as a perceptron. For certain inputs, such as processing raw pixel data, this function results in large fluctuations in outputs with changes to the weights. To adjust for this a different function is utilized, known as a sigmoid function. A sigmoid function is defined as follows:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.3)$$

where $z = \sum w_i x + b$. The advantage of using a function such as a sigmoid is as z grows larger, then $e^{-z} \approx 0$ and thus $\sigma(z) \approx 1$, which limits the effect small changes to weights have on the output [30].

The extent to which humans can influence a neural network beyond the quality or applicability of the external data provided to it is distilled into a category of parameters whose values are set before the learning process begins. These parameters, known as hyperparameters, include the learning rate, data input size, batch size, choice of loss function, and choice of back-propagation mechanism. The learning rate is a parameter which dictates the rate at which a propagation function can change. A large learning rate will converge quickly to a solution at the cost of accuracy, whereas a small learning rate has the potential to result in higher accuracy since corrective steps are incremented more finely. The rate of convergence, however, is often much longer [29].

Loss functions are an error metric used to compare the predicted output of some iteration of the neural network with the “ground truth” and can be continuous or discrete. They are optimized when the derivative of the function is equal to zero and are articulated as follows:

$$J(w) = y - \hat{y} \quad (2.4)$$

The loss function is then back-propagated through the entire network, the derivative of which is used to adjust each neuron’s weight:

$$w^{i+1} = w^i - \frac{\partial}{\partial w^i} J(w) \quad (2.5)$$

A learning rate can also be applied of the form:

$$w^{i+1} = w^i - \eta \frac{\partial}{\partial w^i} J(w) \quad (2.6)$$

More advance techniques for optimizing convergence, especially through the use of adaptive or variable learning rates were employed in this research, such as the use of Adam optimizer for gradient descent optimization and dropout regularization.

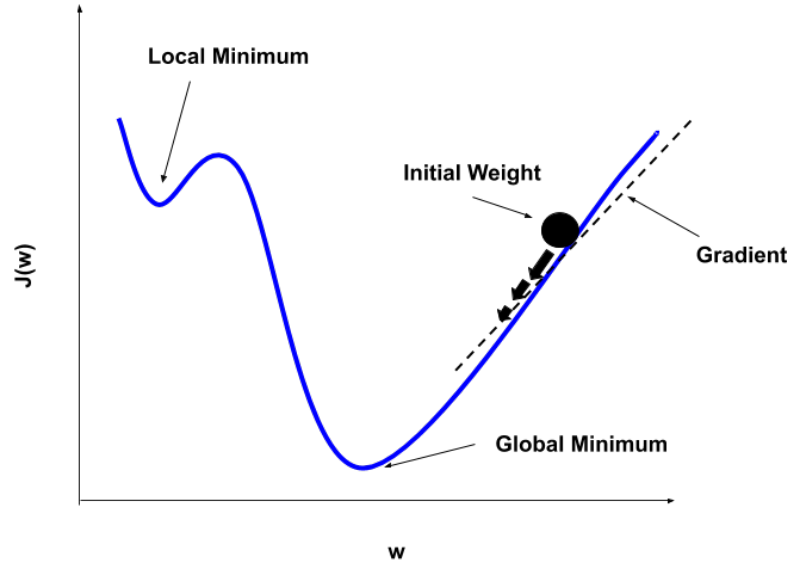


Figure 2.3: Back propagation and convergence. A loss function dependent upon the values of the weights calculated by the neural net, $J(w)$ is optimized using gradient descent until ideally a global cost minimum is found.

2.1.2 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a deep neural network used to process and manipulate image data. Unlike previous algorithms which have largely been replaced by CNN's, very little pre-processing of image data is needed to implement convolutional networks. Instead the networks learn the filters necessary to extract relevant information independent of prior knowledge or human involvement, and, when implemented in a larger architecture can be used for object recognition, detection, and identification [2].

A typical convolutional architecture consists of an input layer of dimension $(number\ of\ images) \times (image\ height) \times (image\ width) \times (color\ depth)$, one to several hidden layers consisting of a convolutional layer followed by a pooling layer, and an output layer.

In a convolutional layer, each unit of a layer receives inputs from a set of units of a previous layer, located in a small subsection of the previous layer's image space. From these localized units, each neuron (node in the network) extracts an elementary visual feature such

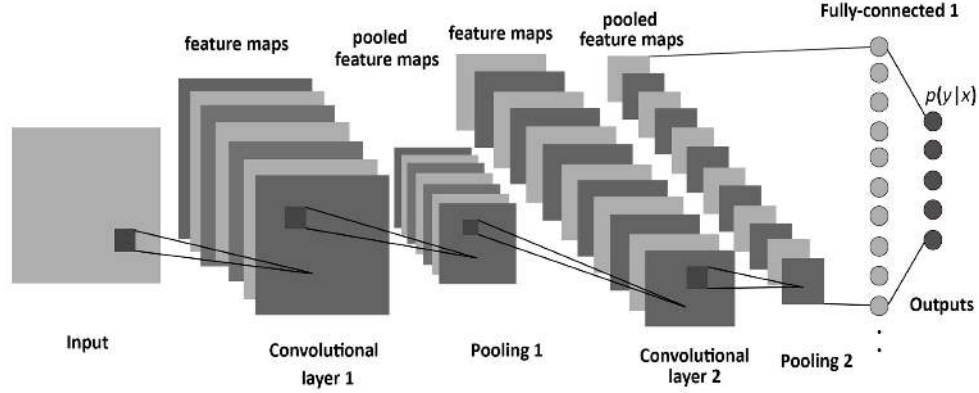


Figure 2.4: Architecture of a convolutional neural network. The network begins with an input layer, usually image data, then extracts feature vectors via convolution. The convolutional layer feeds into a pooling layer to limit the sensitivity to distortions. Finally, the output feature vectors are fed to a fully connected layer where the mapping of convolutional data to desired outputs occur.

as an edge or a corner. These features are then combined by higher layers to output a feature map. At each position, different types of feature maps compute different types of features, a process which is generally performed in parallel, creating multiple planes of neurons where each plane shares a feature map and a single weighted vector. These planes are contained in a single convolutional layer which its relative effect on the overall output is then computed by using self-adjusting weights and biases as well as some activation function, generally a Rectified Linear Unit Function (ReLU) of the form $f(x) = x^+ = \max(0, x)$.

Once a feature has been detected, its exact location becomes of lesser importance than its relative position to other features [31]. Therefore a subsequent layer which performs a local averaging and sub-sampling is employed to reduce the sensitivity of the output to distortions. This is known as the pooling layer.

After some number of combinations of convolution-pooling layer pairs, the network feeds into a fully-connected layer that maps the extracted feature vectors to some output shown in Figure 2.4. A loss function is then applied to the output and the derivative of this result is back-propagated through the network until optimized. For this thesis, the ResNet152 architecture [32] was the primary CNN used. See Figure 2.5 for architectural details.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Figure 2.5: Comparison of various ResNET architectures. ResNET-152 architecture was used for the image processing in this thesis.

2.1.3 Recurrent Neural Networks

Recurrent Neural networks, RNNs, are a class of neural networks used for processing sequential data such as image captioning, speech recognition, machine translation, and video activity recognition [33]. Unlike traditional neural networks, the output from the previous step of an RNN is fed as an input to the current step. Thus it has a hidden state, known as a cell, which remembers information about a sequence. More formally, an RNN takes in some input, x_t and some previous state information, s_{t-1} , to output o_t . The hidden parameters are forward propagated as follows, see Figure 2.6 for more structural detail:

$$s_t = g(w_{ss}s^{t-1} + w_{sx}x^t + b_s) \quad (2.7)$$

Thus, a hidden parameter of the current step is calculated using some activation function, g , which is dependent on the previous state parameter s^{t-1} , scaled by the hidden parameter's weight factor w_{ss} , plus the weight of the input parameter w_{sx} times the input parameter at the current step, x^t , plus a bias term, b_s . Note, all subscripts in the first position signify the parameter that is being calculated and in the second position, represent the parameters the weights are being multiplied by. Similarly, the output is calculated as

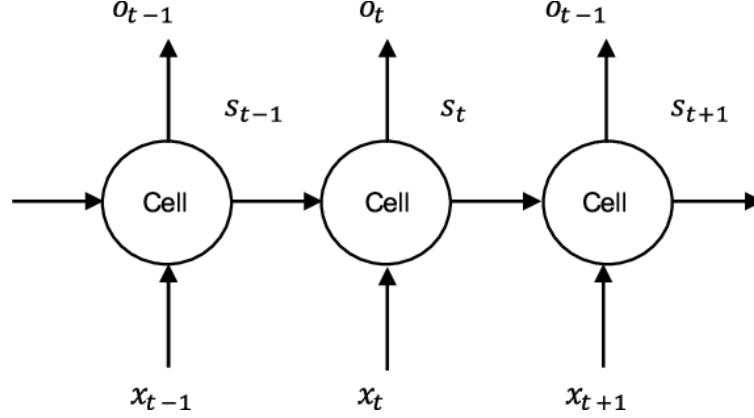


Figure 2.6: Unfolded RNN architecture. Cells, often referred to as “memory cells”, are a function of an input parameter x_t and some previous, hidden parameter, s_t . Cells input parameters are then fed through a $\tanh$ activation function to output some value at the current step, o_t . Image source: [34]

follows:

$$o^t = h(w_{os}s^t + b_o) \quad (2.8)$$

Where h is some other activation function, dependent upon a bias parameter plus the hidden state parameter s^t scaled by some weight w_{os} . Fully expanded, the output at step t then becomes:

$$o^t = h(w_{os}g(w_{ss}s^{t-1} + w_{sx}x^t + b_s) + b_o) \quad (2.9)$$

Common activation functions used for the hidden state parameter are tanh or ReLUs, whereas sigmoid or softmax functions are more common for output parameters and are highly dependent upon the form of the desired output. The loss at every step can be defined as a cross-entropy loss:

$$L^t(o^t, y^t) = -y^t \log(o^t) - (1 - y^t) \log(1 - o^t) \quad (2.10)$$

The loss of the entire sequence can then be defined as:

$$L(o, y) = \sum L^t(o^t, y^t) \quad (2.11)$$

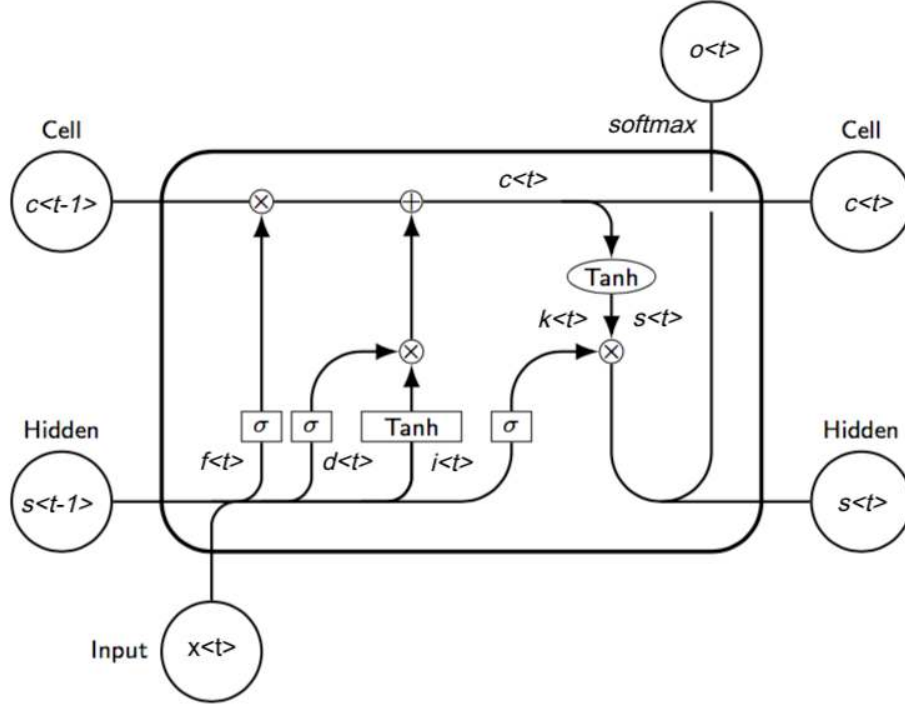


Figure 2.7: Structure of a Long Short-Term Memory, LSTM, block. LSTMs are a function of the previous, hidden input parameter, s_{t-1} , some input parameter x_t , and the previous candidate cell, c_{t-1} . The hidden and input parameters must go through a forget, update, and output gate before producing an output for the current time step, o_t , as well as the next candidate cell, c_t and hidden parameter, s_t .

Like other neural networks, the gradient of the loss functions is back propagated through the network until the loss function is minimized. Unfortunately, with RNNs, the back propagation mechanism is biased towards information closest to the end of the sequence and thus would not accurately model information that was presented early in the sequence. This weakness was addressed by the development of LSTM.

LSTMs, have three gates that use sigmoid activation functions: an update gate, G_{update} , a forget gate, G_{forget} , and an output gate, G_{output} .

$$G_{update} = i^t = \sigma(w_{us}s^{t-1} + w_{ux}x^t + b_u) \quad (2.12)$$

$$G_{forget} = f^t = \sigma(w_{fs}s^{t-1} + w_{fx}x^t + b_f) \quad (2.13)$$

$$G_{output} = k^t = \sigma(w_{ks}s^{t-1} + w_{kx}x^t + b_k) \quad (2.14)$$

Where weights and biases are represented using the same notation delineated earlier, s^{t-1} is the output of the previous LSTM block, and x^t represents the input of the current step. The update gate decides what new information is to be stored in the cell state, the forget gate decides which information to discard, and the output gate dictates the activation value of the final output for the LSTM block at the current step.

The cell “memory” state is defined as follows:

$$c^t = G_{update} * d^t + G_{forget} * c^{t-1} \quad (2.15)$$

Where the candidate for the cell state at step t is calculated as:

$$d^t = \tanh(w_{cs}s^{t-1} + w_{cx}x^t + b_c) \quad (2.16)$$

Thus, at any step, the cell state, c^t decides what to forget, G_{forget} , and information to keep, G_{update} , when updating states. The cell state is then pass through an activation function, $\tanh$, and multiplied with the output gate result to calculate the hidden state:

$$s^t = G_{output} * \tanh(c^t) \quad (2.17)$$

The hidden state from the current LSTM block step is passed through a softmax activation to obtain the predicted output, o^t , of the current block. Loss functions and back propagation are used in similar manner as previously outlined to optimize the network.

Finally, two LSTM networks can be trained, one as an encoder, to process an input sequence then return its own internal state, and one as a decoder, to interpret the encoder’s output and reproduce the encoder’s input. The encoder’s output will later serve as “context” for the navigation agent’s interpretation of instructions, while the decoder will provide a

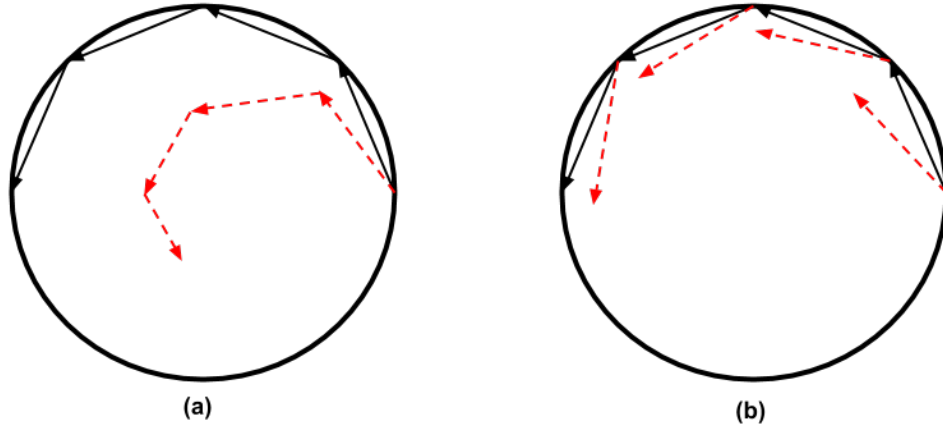


Figure 2.8: Example of corrective action in teacher forcing. (a) Without teacher forcing, network predictions can deviate greatly from what is expected. (b) With teacher forcing, network predictions can be corrected at every step. Image source: [35]

metric to compare the efficacy of the encoder.

2.1.4 Teacher Forcing vs Student Forcing

Teacher forcing is a strategy for training LSTMs that uses actual or expected model output from the prior step as an input for the current step instead of output generated by the network [29]. Thus instead of calculating and summing activation functions from the current input and hidden input parameters, x_t and s_t respectively, the ground truth, or predicted output from the previous step is used. In following this method, steps can be decoupled for comparing loss functions to predicted outputs at each step and training can be made parallel. Furthermore, if the model's calculated weights are erroneous, teacher forcing can act as a fail-safe, providing corrective steps based on ground truth at every step. See Figure 2.8. Since the outputs of teacher forcing are probabilistic by nature, however, teacher forcing is limited if generated sequences vary greatly from what is seen during training phase.

Similar to teacher forcing, student forcing, or more accurately, best student forcing, is a procedure for training LSTMs with output-to-hidden recurrence. Best student forcing is a

simple adversarial training mechanism that uses generated sequences as temporary ground-truths [36]. By using local information generated as the output, the mechanism becomes more robust to generalizations, especially on previously unseen and varying data.

2.2 Language Augmentation and Instruction

A language interpretation model was used with a LSTM-based, recurrent neural network architecture. In such a model an agent is initialized with a route, r_0 , and a vectorized set of command instructions $\langle w_1, w_2, w_3, \dots, w_N \rangle$ where w_i is a single tokenized word and N is the length of the instruction. In alignment with Sutskever et al, [37, 15], the LSTM encoder was sequentially provided each word, w_i , in reverse order from each vectorized command.

To allow better navigation through unknown environments, the agent was also generalized by training an encoder LSTM on a finite number of navigation instructions, with a decoder LSTM attached to the output. The LSTM produced a decoded set of instructions that was compared to the input instructions using a word-word co-occurrence probability statistic known as BLEU, and optimized using a weighted least-squares during training phase. During the training phase of the navigation agent, instructions were then fed through the encoder portion of the LSTM, replacing the original input instruction. This process will be referred to from now on as instructional augmentation.

2.3 Action-In-Image Space

The primary sequence-to-sequence agent in [15] utilizes a limited visuomotor control, allowing a field of vision of 30° , and only perceiving visual input centered in front of the agent. This presents challenges for instruction-following specifically when the instruction relates a navigation instruction relative to an object. Consider the following command: “*go straight through the kitchen then turn left at the window*”. The action of “*turning left at the window*” requires the agent to perform a series of turning actions until it sees the window in the center of view. The agent then needs to return to its original orientation, and turn

left to fully complete the task. Fried et al. [10] proposed directly allowing the agent to make decisions about its action space by using a panoramic representation, mapped from the lower-level visuomotor control instead.

The panoramic representation would thus require a physical agent to first obtain a 360-degree panoramic view of its surroundings from its current location (implemented as 36 different view angles: 12 orientations with three elevations taken at 30 degree intervals). However, for the data set used, panoramic images were readily available and the mapping from visual space to action space was already developed by Fried [10].

Utilizing the augmented panoramic method, each observed image was fed through a ResNet-152 [32] CNN architecture, pre-trained on ImageNet [2], to produce a mean-pooled feature vector encoded with navigable direction data.

Next, analogous to the embedded word instructions, a feature was learned for each action taken in the panoramic image space. The encoded image and previous action feature were then concatenated together to form a single action vector, a_t , consisting of positional data, navigable direction data, and action performed. A decoder LSTM was used to optimize the action-in-image-space, $L_t = LSTM_{dec}(a_t, L_{t-1})$.

2.4 Navigation Prediction

The action prediction mechanism was a multilayer perceptron that utilized a softmax activation function. Given instructional and current state information, the model would generate a probability distribution over all available actions in the action space, then select the most likely action from the distribution. Unlike other models [10], this model did not use increased attention mechanisms or hidden states to contextualize instruction vectors since it is believed that the instruction augmentation acted already to contextualize actions within the image space.

CHAPTER 3

METHODOLOGY

3.1 The Data Set

For machine learning tasks in such fields as computer vision, augmented reality, and robotic navigation, it is often necessary for training purposes to draw upon a large corpus of data. Specifically for language based undertakings, the task of navigation through unknown environments using human-like directions is difficult to generate due to the necessity of human involvement. The lack of machine’s ability to create human-like instructions discards massive potential data sets such as those derived from Google Maps or those used for alternative tasks like autonomous driving, for example, Waymo’s Open Data Set [38] or Berkeley’s DeepDrive data set [39]. Furthermore, data sets derived from video sequences, which could dually provide image data and instructional data, are often limited in scope and quality, leading to incomplete or blurry views.

An ideal data set for navigation would thus be one that is indoors, embedded with depth metrics, high in picture quality, continuous in viewpoint, and broad in diversity of scenery. For the instructional navigation task specifically, this data set would also need to include high quality human-like instructional inputs. Matterport 3D’s Room-to-Room (R2R) is a large-scale RGB-Depth Image (RGB-D) indoor data set containing 10,800 panoramic views from 194,400 RGB-D images of 90 building-scale scenes [40].

In the R2R data set, panoramic views are distributed throughout the entirety of the navigable paths and are separated an average 2.25 meters. Each viewpoint is comprised of 18 RGB-D images captured from a single 3D position at approximately the height of a standing person and annotated with six camera position parameters [15]. The data set also includes meshes, room segmentation, and objects and have diverse scenes, including

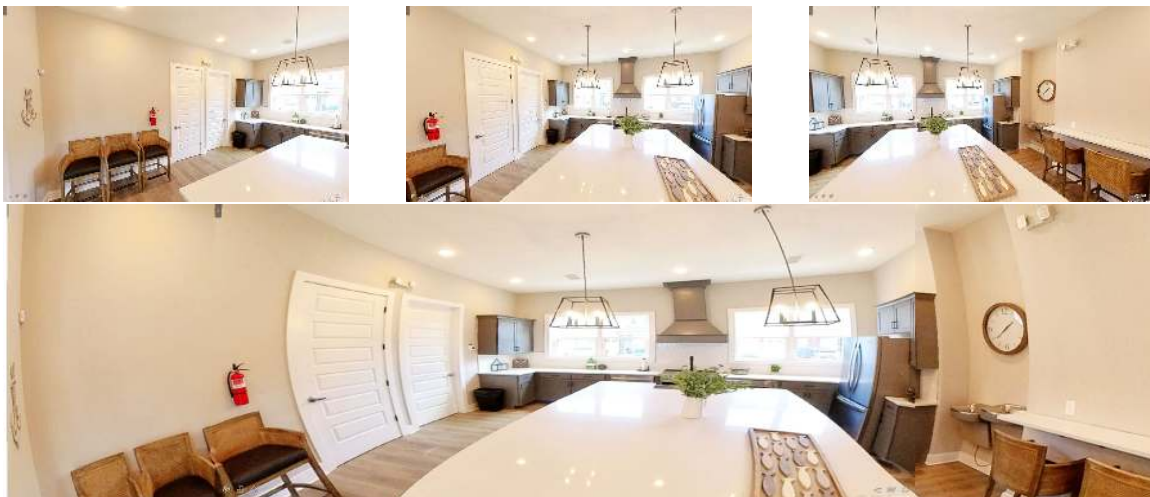


Figure 3.1: Example of a Matterport 3D scene both with and without panoramic view. In the original data set for the room-to-room navigation task, all images are stitched together to create a panoramic image. The panoramic image is then embedded with navigational path data as well as camera position data.

houses, offices, hotels, and apartments of varying sizes.

3.2 Experimental Setup

This investigation utilizes the structure developed by [15, 10] with the exception of using augmented instructions, pre-contextualized for the agent’s navigation. A visual network was used based on ResNET152, a language interpretation model was developed using LSTM, and a sequence-to-sequence navigation mechanism was developed using both teacher and student forcing techniques.

3.2.1 Training

Two navigation agents were trained using both teacher-forcing and student-forcing techniques [10, 15]. The desired output of the network was the action which resulted in the shortest-path given the navigation agent’s current state, s_t , and the navigation goal. A cross entropy loss was used on each network to maximize the predicted action, a^t , given the previous state-action sequence. At each step during training the ground-truth for the target action was supplied during teacher-forcing. This obviates the limitations of the tech-



Figure 3.2: A dollhouse model example of a navigable scenery created by Matterport 3D. In these models, meshes are rendered as well as room segmentation to allow for easier object and room identification. [40].

nique when attempting to generalize navigation agents to unknown environments, where no ground truth exists. A student forcing technique was also employed, allowing the agent to output a probability distribution to sample the next action from.

3.2.2 Implementation

All textual data was first converted to lower-case, tokenized, and filtered for words that did not occur more than twice. A baseline was found using the human-generated instructions for routes present in the R2R data set. For tests exploring the impact of augmented instructional commands, the textual data was then fed through an encoder network that extracted only interesting instruction features, allowing for an encoding schema to be used to produce new, natural instruction commands. The word embedding size was limited to a maximum input length of 75 words. 2,000 new instructional commands were then produced, shuffled into the existing data set for the instructional augmentation test.

Each LSTM had a feature size of 2048 with a dropout rate of 0.3 and a hidden feature space of 512 layers. The size of the input action embedding was set to 28 and the size of

the input word embedding was set to 300.

For all cases, a train-test split of 80/20 was applied to the data set. All training was performed in PyTorch using Adam optimizer with a weighted decay of 0.0005, a learning rate of 0.0001, and a batch size of 20. For all cases, 20,000 iterations were found to be sufficient for convergence.

3.3 Evaluation Metric

3.3.1 Bleu: Bilingual Evaluation Understudy

BLEU is a metric for evaluating a generated sentence to some reference text, in this case, the Sequence-to-Sequence (S2S) encoder/decoder pair for translating input instructions into contextualized input instructions. Developed for evaluating predictions made by neural network driven machine translation systems, BLEU is computationally inexpensive and quick to calculate, widely adopted in the machine translation community, and correlates highly with human evaluation [41].

BLEU uses a modified form of precision to score a candidate translation. For each word in the candidate translation, the algorithm takes its maximum total count in any reference translation as its occurrence value, then divides by the total number of distinct words in the candidate sentence[41]. For example, consider the following two reference instructions:

(1) *“Go down the hall and through the door. Stop at the bed”*

(2) *“Go down the stairs. Make a left at the bottom of the stairs. Stop at bottom.”*

Now consider the following augmented instruction:

(*) *“Go stairs. Stop at door. Right at door.”*

The word “go” occurs once in both references and gets a score of 1 out of 8 distinct words. The word “stairs” occurs once in the augmented instruction example and gets a score of 1 out of 8, though it could have had a maximum score of 2 out of 8 since it appears twice in

reference sentence (2). Using similar logic as the word “*go*”, “*stop*” achieves a score of 1 out of 8. The word “*door*” appears twice in the augmented instruction, but only appears a maximum of once in the reference sentences. Therefore, the word “*door*” also has a score of 1 out of 8. The word “*at*” appears twice in our augmented sentence and also twice in reference sentence 2. It receives a score of 2 out of 8. Finally, the word “*right*” appears once in our augmented sentence but zero times in our reference sentences. It receives a score of 0 out of 8. Thus, the BLEU score for our augmented sentence is:

$$P = \frac{1 + 1 + 1 + 1 + 2 + 0}{8} = \frac{3}{4}$$

3.3.2 Navigation Metrics

The navigational agent’s performance was measured using evaluation protocols integrated into the R2R data set. Defined as the shortest path distance from the agent’s final position and the goal location, *navigation error* is the only metric that was absolute in value and was thus normalized to a 13 m planned path in the results section.

The *success rate* was based on the final location of the agent. A particular run was considered successful if the agent stopped within 3 m or less of the navigational goal. This threshold was dictated by the magnitude of error of approximately one visual frame.

Though the stopping requirement for the navigation agent was predicated upon the recognition of the goal location by the agent and though it is agreed that this is a fundamental aspect to completing the task [15], another metric was employed to track the success of the agent separate from its ability to recognize its stopping location: *oracle success*. The oracle success rule measures the closest point the navigation agent achieved relative to the goal along the entire length of its trajectory and was employed by many previous works [15, 10, 7]. The oracle success rate is then determined using the same cutoff criterion as the success rate metric.

CHAPTER 4

RESULTS AND ANALYSIS

4.1 Language Training Results

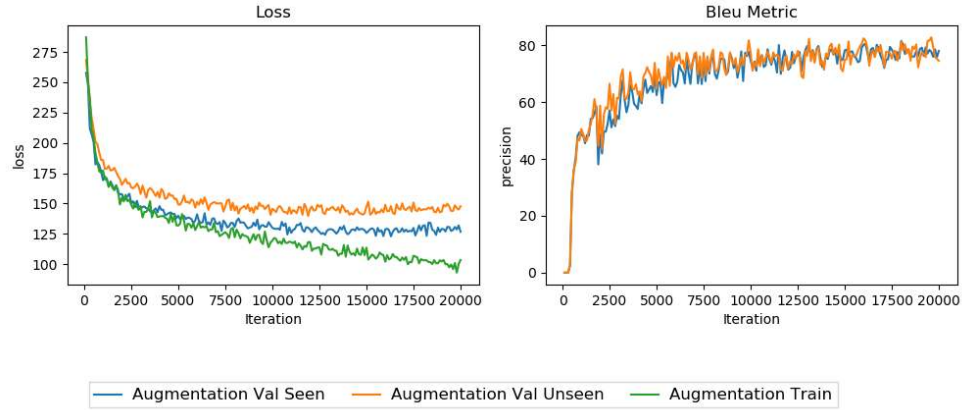


Figure 4.1: Loss function and BLEU metric score, measured in percent precision, for the instruction augmentation network.

The language augmentation network was trained for 20000 iterations with a change loss of 63% and converging on the validation seen and unseen loss at approximately 52.7% and 43.6% change loss. The network generalized well, producing a BLEU score of approximately 80% for both seen and unseen validation sets. This implies a high correlation between the original input and the encoded/decoded input, even with the constraint of encoded/decoded instructions being compressed to a length of 75 at the output of the encoder then expanded back to a maximum length of 300 at the output of the decoder.

4.2 Navigation Results

Table 4.1 summarizes the methods explored in this thesis with two baseline methods, random search, results procured by [15], and a greedy search algorithm which chooses the

optimal action at each step. The random search performed worst, with a navigation error on seen values of 9.45 m and success rate of 15.9%. The generalization from seen to unseen data was not expected, outperforming seen data with a navigation error of 9.23 m, success rate of 16.3%, and oracle success rate of 22.0%. The increase in performance for all three metrics would imply a method which adapts better to previously unseen trajectories than those seen, however, the results are likely due to statistical fluctuation. This method was robust when evaluating it to previously unseen test data, scoring a navigation error of 9.7 m and success rate of 13.2%.

Slightly more sophisticated, the greedy search algorithm obtained navigation errors for seen and unseen trajectories of 7.11 m and 7.67 m respectively, with a success rate of 38.3% for previously seen trajectories and 33.2%. This method did not generalize well, scoring a navigation error of 8.2 m and a success rate of 17%.

The teacher-forcing method developed in this thesis is the first method to provide a baseline for comparing the efficacy of the instructional augmentation algorithm implemented. Without augmentation, the teacher-forcing method scored a navigation error on seen, unseen, and test sets of 6.23 m, 6.26 m, and 6.80 m accordingly, and success rates of 48.1%, 41.0%, and 23.1%, the highest of the baseline methods. After implementing the instructional augmentation algorithm, navigation error dropped by approximately 2.7 m for previously seen environments, 1.14 m for previously unknown environments and scores an overall success rate of 35.5% for the test set, 12.4% higher than when augmentation was not used.

The student-forcing method was found to generalize better than the teacher-forcing method from known to unknown environments. The baseline method scored a navigation error of 6.27 m, 7.28 m, and 7.55 m for seen, previously unseen, and test environments. The method also achieved a success rate of 42.7% in the seen validation regime, 30.6% in the unseen validation regime, and 19.1% on the test set.

Though the baseline method of teacher-forcing outperformed the baseline method of

student-forcing, after instructional augmentation was applied, student-forcing outperformed teacher-forcing on all fronts, scoring the highest success rate of all the methods, 46.1%, with the lowest navigational error 5.02 m. One possible explanation for this is due to the mechanics of teacher-forcing using “ground truths, which bypass the instructional augmentation model in early stages, propagating bias throughout the entirety of the model. Student-forcing, however, uses a mechanism analogous to “best guessing and thus makes decisions at every step based off information obtained from the instructional augmentation model. This does not imply, however, that student-forcing is superior to teacher-forcing in the navigation regime, rather that student-forcing generalizes better to unknown environments when using augmented instructional inputs.

Method	Validation Seen			Validation Unseen			Test	
	NE	SR	OSR	NE	SR	OSR	NE	SR
Random Search [10]	9.45	15.9	21.4	9.23	16.3	22.0	9.70	13.2
Greedy Search	7.11	38.3	49.6	7.67	33.2	47.1	8.20	17.0
Teacher Forcing	6.23	48.1	61.5	6.26	41.0	53.5	6.80	23.1
Teacher Forcing IA	3.57	67.9	74.0	5.12	48.4	67.0	5.47	35.5
Student Forcing	6.27	42.7	53.7	7.28	30.6	52.6	7.55	19.1
Student Forcing IA	3.28	68.8	76.3	4.97	50.1	65.7	5.02	46.1

Table 4.1: Summary of Results of Methods. As baselines, both random and greedy searches were used on identical validation/test sets as the teach-forced/student-forced models as well as the augmented instruction teacher-forced/student-forced models. Metrics collected included navigation error (NE), in meters, success rate (SR), (with a cutoff of 3 m from goal), and oracle success rate (OSR), (cutoff of 3 m from goal at any point in the trajectory). A low navigation error is more desirable, whereas high success rates and oracle success rates are more desirable.

CHAPTER 5

COMPARISON TO PRIOR WORK

Previous work utilizing the R2R data set have largely focused on the development of deep learning methods to improve the performance of navigation agents. Notable among these are included in the following sections and include methods which implement S2S networks for language interpretation and pragmatic inference, as well as deep reinforced learning for path planning mechanisms. The most successful of these, however, developed both a deep learning navigation agent as well as utilized visual data augmentation.

Method	Validation Seen			Validation Unseen			Test	
	NE	SR	OSR	NE	SR	OSR	NE	SR
RPA [7]	5.56	42.9	52.6	7.65	24.6	31.8	7.53	25.3
BMAS [15]	6.01	38.6	52.9	7.81	21.8	28.4	7.85	20.4
Speaker-Follower [10]	3.08	70.1	78.3	4.83	54.6	65.2	4.87	53.5
Human [15]	—	—	—	—	—	—	1.61	86.4
Results of this Thesis	3.28	68.8	76.3	4.97	50.1	65.7	5.02	46.1

Table 5.1: Comparison to Other Work. The human baseline is a series of human-generated path plans, whereas the remaining methods are all machine learning based path planning models. RPA and BMAS are both models which emphasize navigation model development whereas the speaker-follower model utilized data input augmentation as well as navigation model development.

5.1 Navigation Models

5.1.1 Human Baseline

As an added baseline, a human-generated path plan was collected by Anderson, et al [15]. The human navigation agents were provided with navigation instructions, same as

the robotic navigation agent, and were allowed to explore the environment in the form of first-person clickable viewpoints. They were then asked to then navigate to the stop goal. The quality of data was ensured by [15] as they paid the human navigation agents bonuses for stopping within 3 meters of the true goal location.

The human baseline represents the epitome of what robotic navigation based on natural language could achieve. With a navigation error of 1.61 m and a success rate of 86.4%, it is by far the highest performing method. The complexity of the visual, linguistic interpretation, and decision-making processes used in the human baseline are impossible to fully articulate, however contributing factors to it's success include a human's ability to abstract both visual input and language inputs, a human's ability to make inferences in the decision making process, and a human's advantage in communication, namely the ability to gesture and engage in dialog.

Our top performing result in comparison to the human baseline achieved a success rate of over 40% less than the human success rate with a navigation error of over three times the amount of the human's. The top navigation result overall achieved comparable results, with a 33% difference between the success rate of the two and a navigation error difference of 3.26 m.

5.1.2 Bring Me A Spoon

The BMAS authors are the proprietors of the R2R data set and the first to implement a S2S model using the aforementioned data set [15]. Their top performing model was a student-forcing S2S network that had a navigation error and success rate of 6.01 m and 38.6% in the known environment regime and a navigation error and success rate of 7.85 m and 20.4% in the unknown environment regime. The success rate being halved from known to unknown is demonstrative of the difficulty to generalize methods to unknown environments. This method did not use augmentation of any kind.

5.1.3 Reinforced-Learning for Planned-Ahead Navigation

The authors of RPA method summarized in Table 5.1 utilized a combination of model-free and model-based deep reinforced learning to bridge the gap between synthetic instructions and real-world navigation tasks, specifically targeting the R2R physical data set. They successfully demonstrated their model’s scalability and ability to generalize to unseen environments in [7], slightly outperforming BMAS’s scores.

5.2 Augmentation Models

5.2.1 Speaker-Follower Model

The speaker-follower model developed by [10], utilizes a speaker-annotated route selection to generalize better to navigation in new environments. At test time, the speaker is used to “re-score” the routes selected by the follower, allowing the speaker to decide how well the route explains the navigational direction, providing pragmatic context for navigation instructions. Also utilized in this approach was migrating the visuomotor control space to the panoramic representation provided in the R2R data set, allowing decisions to be made in a more complete visual environment.

This model performs the best above all previous and current work. By augmenting both the visual mechanics and the language interpretation mechanics, Fried et al. were able to achieve a navigation error of 3.08 m and success rate of 70.1% in known environment regimes. Their method also generalized well to unknown environment regimes, with a navigation error of 4.87% and a success rate of 53.5%. These results best support the approach chosen in this thesis: focus on developing a navigation method with emphasis towards augmenting input parameters, both visual and linguistical, and interpretation of data to best map human-to-machine translations.

CHAPTER 6

CONCLUSION

In conclusion, an attempt to better understand the effect of natural language for visual-and-language navigation agents was made through the development of language augmentation networks fed into navigation LSTM networks. Using teacher-forced and student-forced sequence-to-sequence models, baselines were established with navigation errors of 6.23 m and 6.27 m on seen data and generalizing to unseen data with navigation errors of 6.26 m and 7.28 m. Instruction augmentation was then implemented, with an improvement of navigation errors by roughly 3 m for each baseline in the seen regime and 2.31 m for the best performing method, student-forced navigation agent with instructional augmentation, in the unseen regime. Compared to other works, this method outperformed early vision-and-language agents on the same data set, as well as some deep reinforced learning agents. Compared with more contemporary vision-and-language agents [10], the navigation agent developed in this thesis performed slightly worse, but still comparable with a 7% margin of success rate between the two on unseen data. This result however, still demonstrates that using semi-supervised methods with input data augmented results in better navigation scores compared to those methods that do not augment their input data. Thus using data augmentation is beneficial for navigation agents when generalizing from known to unknown environments.

6.1 Future Work

As has been established through the research conducted in this thesis, data augmentation and semi-supervised learning methods are effective when generalizing navigation agents from seen to unseen environments. Using similar augmentation methods on visual data is one possible path of further development for improving and generalizing navigation agents.

In this method, a sequence-to-sequence encoder-decoder pair could be used to allow the robot to abstract perceived relevant objects related to path instructions. This modification is believed to lead to a better grounding for natural language commands and logically would narrow the gap between the success rate and the oracle success rate in both seen and unseen environments and improving the navigation error of the agent from its end goal.

Another research topic worth pursuing is supplanting the navigation algorithms developed here to a physical robot, then deploying the robot to train on real-life inputs through sensors. The room-to-room data set largely lacked extraneous obstacles or challenges that might exist in the physical world that deployment on a physical robot would encounter. For example, there are no people or animals in the room-to-room data set, adding a challenging layer of complexity to an already complicated problem. Furthermore, environments in the room-to-room data set are static which is not an accurate rendering of many environments in the physical world. A robot generalized from the simulated environments used to the physical world would thus have to learn to traverse a changing environment with doors closing and people moving constantly.

Appendices

APPENDIX A

NAVIGATION EXAMPLE



(a) Action: Move Forward.



(b) Action: Turn Right.



(c) Action: Move Forward.



(d) Action: Move Forward.



(e) Action: Move Forward.



(f) Action: Turn Right.



(g) Action: Turn Right.



(h) Action: Move Forward.



(i) Action: Move Forward.



(j) Action: Turn Right.



(k) Action: Turn Right.



(l) Action: Move Forward.



(m) Action: Turn Left.



(n) Action: Move Forward.



(o) Action: Stop.

Figure A.1: Example path of the following instruction: “Go up the stairs and turn right. Enter the first door on the left. Stop at the bathtub.”

REFERENCES

- [1] A. Singh, L. Yang, C. Finn, and S. Levine, “End-To-End Robotic Reinforcement Learning without Reward Engineering,” 2019.
- [2] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “ImageNet Large Scale Visual Recognition Challenge,” *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015.
- [3] J. Zeng, R. Ju, L. Qin, Y. Hu, Q. Yin, and C. Hu, “Navigation in unknown dynamic environments based on deep reinforcement learning,” *Sensors (Switzerland)*, vol. 19, no. 18, 2019.
- [4] C. P. Lam, C. T. Chou, K. H. Chiang, and L. C. Fu, “Human-centered robot navigation-towards a harmoniously human-robot coexisting environment,” *IEEE Transactions on Robotics*, vol. 27, no. 1, pp. 99–112, 2011.
- [5] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” *Springer Tracts in Advanced Robotics*, vol. 70, no. STAR, pp. 3–19, 2011.
- [6] L. Tai, J. Zhang, M. Liu, J. Boedecker, and W. Burgard, “A Survey of Deep Network Solutions for Learning Control in Robotics: From Reinforcement to Imitation,” no. December, 2016.
- [7] X. Wang, W. Xiong, H. Wang, and W. Y. Wang, “Look Before You Leap: Bridging Model-Free and Model-Based Reinforcement Learning for Planned-Ahead Vision-and-Language Navigation,” *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 11220 LNCS, pp. 38–55, 2018.
- [8] C. Y. Ma, J. Lu, Z. Wu, G. AlRegib, Z. Kira, R. Socher, and C. Xiong, “Self-monitoring navigation agent via auxiliary progress estimation,” *7th International Conference on Learning Representations, ICLR 2019*, pp. 1–18, 2019.
- [9] H. Tan, L. Yu, and M. Bansal, “Learning to Navigate Unseen Environments: Back Translation with Environmental Dropout,” pp. 2610–2621, 2019.
- [10] D. Fried, R. Hu, V. Cirik, A. Rohrbach, J. Andreas, L.-p. Morency, T. Berg-kirkpatrick, K. Saenko, D. Klein, and T. Darrell, “Speaker-Follower Models for Vision-and-Language Navigation,” no. Figure 1, pp. 1–25, 2018.

- [11] T.-C. Chi, M. Eric, S. Kim, M. Shen, and D. Hakkani-tur, “Just Ask: An Interactive Learning Framework for Vision and Language Navigation,” 2019.
- [12] M. MacMahon, B. Stankiewicz, and B. Kuipers, “Walk the talk: Connecting language, knowledge, and action in route instructions,” *Def*, vol. 2, no. 6, p. 4, 2006.
- [13] V. Blukis, D. Misra, R. A. Knepper, and Y. Artzi, “Mapping Navigation Instructions to Continuous Control Actions with Position-Visitation Prediction,” no. CoRL, 2018.
- [14] D. L. Chen and R. J. Mooney, “Learning to interpret natural language navigation instructions from observations,” *Proceedings of the National Conference on Artificial Intelligence*, vol. 1, pp. 859–865, 2011.
- [15] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sunderhauf, I. Reid, S. Gould, and A. Van Den Hengel, “Vision-and-Language Navigation: Interpreting Visually-Grounded Navigation Instructions in Real Environments,” *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 3674–3683, 2018.
- [16] L. Ke, X. Li, Y. Bisk, A. Holtzman, Z. Gan, J. Liu, J. Gao, Y. Choi, and S. Srinivasa, “Tactical rewind: Self-correction via backtracking in vision-and-language navigation,” in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2019-June, 2019, pp. 6734–6742, ISBN: 9781728132938.
- [17] C. Y. Ma, Z. Wu, G. Alregib, C. Xiong, and Z. Kira, “The regretful agent: Heuristic-aided navigation through progress estimation,” *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2019-June, pp. 6725–6733, 2019.
- [18] M. A. Goodrich and A. C. Schultz, *Human-robot interaction: A survey*, 2007.
- [19] L. S. Lopes and A. Teixeira, “Human-robot interaction through spoken language dialogue,” in *IEEE International Conference on Intelligent Robots and Systems*, vol. 1, 2000, pp. 528–534, ISBN: 0780363485.
- [20] D. Spiliotopoulos, I. Androutsopoulos, and C. D. Spyropoulos, “Human-robot interaction based on spoken natural language dialogue,” in *Proceedings of the European Workshop on Service and Humanoid Robots*, vol. 1, no. August, pp. 25–27, 2001.
- [21] T. Fong, C. Thorpe, and C. Baur, “Collaboration, Dialogue, and Human-Robot Interaction,” pp. 255–256, 2003.
- [22] J. Thomason, A. Padmakumar, J. Sinapov, N. Walker, Y. Jiang, H. Yedidsion, J. Hart, P. Stone, and R. J. Mooney, “Improving grounded natural language understand-

- ing through human-robot dialog,” *Proceedings - IEEE International Conference on Robotics and Automation*, vol. 2019-May, pp. 6934–6941, 2019.
- [23] J. Thomason, M. Murray, M. Cakmak, and L. Zettlemoyer, “Vision-and-Dialog Navigation,” no. Section 4, 2019.
 - [24] F. Ricci and L. Rokach, *Recommender Systems Handbook*, 2nd. Springer, 2015, ISBN: 9781489976369.
 - [25] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” *Advances in Neural Information Processing Systems*, vol. 2017-Decem, no. Nips, pp. 4300–4308, 2017.
 - [26] K. Subramanian, C. L. Isbell, and A. L. Thomaz, “Exploration from demonstration for interactive reinforcement learning,” in *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS*, 2016, pp. 447–456, ISBN: 9781450342391.
 - [27] S. Tellex, R. Knepper, A. Li, D. Rus, and N. Roy, “Asking for Help Using Inverse Semantics,” 2015.
 - [28] R. A. Knepper, S. Tellex, A. Li, N. Roy, and D. Rus, “Recovering from failure by asking for help,” *Autonomous Robots*, vol. 39, no. 3, pp. 347–362, 2015.
 - [29] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
 - [30] M. A. Nielsen, *Neural Networks and Deep Learning*, Jun. 2018.
 - [31] Y. LeCun and Y. Bengio, “Convolutional Networks for Images, Speech, and Time-Series,” vol. 4, no. April 2016, pp. 2571–2575, 1995.
 - [32] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2016-Decem, 2016, pp. 770–778, ISBN: 9781467388504.
 - [33] A. Ullah, J. Ahmad, K. Muhammad, M. Sajjad, and S. W. Baik, “Action Recognition in Video Sequences using Deep Bi-Directional LSTM with CNN Features,” *IEEE Access*, vol. 6, pp. 1155–1166, 2017.
 - [34] Y. Zuo, Y. Wu, G. Min, and L. Cui, “Learning-based network path planning for traffic engineering,” *Future Generation Computer Systems*, vol. 92, pp. 59–67, 2019.
 - [35] S. Srihari, “Deep Learning Srihari RNNs: Teacher Forcing,”

- [36] J. Sauder, T. Hu, X. Che, H. Yang, and C. Meinel, “Best Student Forcing : A Simple Training Mechanism in Adversarial Language Generation,” no. May, pp. 4680–4688, 2020.
- [37] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” *Advances in Neural Information Processing Systems*, vol. 4, no. January, pp. 3104–3112, 2014.
- [38] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, V. Vasudevan, W. Han, J. Ngiam, H. Zhao, A. Timofeev, S. Ettinger, M. Krivokon, A. Gao, A. Joshi, Y. Zhang, J. Shlens, Z. Chen, and D. Anguelov, “Scalability in Perception for Autonomous Driving: Waymo Open Dataset,” pp. 2443–2451, 2020.
- [39] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, “BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning,” pp. 2633–2642, 2020.
- [40] A. Chang, A. Dai, T. Funkhouser, M. Halber, M. Niebner, M. Savva, S. Song, A. Zeng, and Y. Zhang, “Matterport3D: Learning from RGB-D data in indoor environments,” in *Proceedings - 2017 International Conference on 3D Vision, 3DV 2017*, 2018, pp. 667–676, ISBN: 9781538626108.
- [41] K. Papineni, S. Roukos, T. Ward, W.-j. Zhu, and Y. Heights, “IBM Research Report Bleu : a Method for Automatic Evaluation of Machine Translation,” *Science*, vol. 22176, no. February, pp. 1–10, 2001.